

Deterministic Dual-ROI and LiDAR-Scheduled Architectures for Autonomous Obstacle Avoidance: A Hybrid Edge-Computing Approach

Le Thanh Tung

FPT University

thanhtung22012006@gmail.com

Huynh Nhat

FPT University

huynhnhatphuyentuyhoa@gmail.com

Lo Thai Quyen

FPT University

quyenlothai@gmail.com

Nguyen Trung Tin

FPT University

trungtin1218@gmail.com

Nguyen Duc Bao Lam

FPT University

lam01662052827@gmail.com

Abstract

Autonomous racing on edge devices requires optimized algorithms rather than heavy deep learning models. This proposal presents a deterministic navigation architecture based on Dual Region of Interest (ROI) vision, LiDAR scheduling, PID control, and Finite State Machine (FSM)-based decision making. The system consists of three modules: Approach 1 performs orthogonal (90°) lane-switching using LiDAR and distal ROI feedback; Approach 2 applies trigonometric LiDAR-based evasion for smoother obstacle avoidance; and Approach 3 handles Smart City intersections through traffic sign recognition, traffic light prioritization, and event-driven FSM transitions. By separating continuous lane tracking from discrete safety and intersection behaviors, the architecture aims to improve real-time performance and control reliability on edge-computing platforms for the Jetson AI Racer Challenge.

Keywords: Autonomous Vehicles, Dual ROI, LiDAR Scheduling, Orthogonal Evasion, PID Control, Edge AI.

1 Introduction & Motivation

Real-time navigation in autonomous miniature vehicles requires a delicate balance between perception accuracy and computational efficiency. The motivation for this research stems from the hardware constraints of edge computing platforms (e.g., NVIDIA Jetson), where executing complex semantic segmentation networks often leads to severe frame rate drops (FPS) and thermal throttling. Consequently, this study proposes an optimized deterministic methodology relying on traditional geometric computer vision (CV) and Light Detection

and Ranging (LiDAR) telemetry to achieve high-speed, robust autonomous racing.

1.1 Problem Statement & Research Questions

The primary challenge is safe obstacle evasion while maintaining stable lane tracking under edge-computing constraints. This requires a reliable perception-to-control pipeline that detects obstacles using LiDAR and camera data, estimates their spatial positions, and triggers deterministic control commands. This proposal addresses the following research questions:

1. How can Bird's Eye View (Perspective Transform) and Dual-ROI configurations reduce the vulnerabilities of traditional HSV color masking in lane tracking?
2. How can the vehicle effectively bridge object detection and autonomous control by mapping real-time LiDAR proximity data to specific scheduled evasion maneuvers?
3. Can a LiDAR-triggered Finite State Machine (FSM) utilizing orthogonal or trigonometric trajectories outperform continuous neural network steering in obstacle avoidance scenarios?

2 Related Work

Current approaches to autonomous edge navigation generally fall into two categories: **Deep Learning (End-to-End & Segmentation):** Models like ResNet-18 or YOLOv8 offer robust generalization [1]. However, even with TensorRT quantization, they consume significant VRAM and struggle with deterministic state transitions during sudden obstacle encounters. **Traditional CV, PID Control & Sensor-Based Navigation:**

Lightweight computer vision remains a practical solution for small-scale autonomous robots because it can support real-time lane tracking with lower computational cost. Shamoushaki and Ghasemi applied Gaussian Blur, masking, and image moments for line detection in differential-drive mobile robots, showing that centroid-based visual features can support lane detection, lane changing, obstacle avoidance, and laboratory validation [5]. Similarly, Farkh and Aljaloud proposed a vision-navigation line tracking robot where the error between the camera frame center and the detected line contour center is used as input to a PID controller for steering and speed control [6]. LiDAR clustering (e.g., DBSCAN) provides excellent distance metrics but lacks the semantic awareness needed to identify lane boundaries [2]. Real-time LiDAR-based obstacle avoidance methods further demonstrate that local sensor information can be used to select feasible sub-goals and update trajectories during motion, enabling robots to respond to unexpected obstacles without requiring a pre-built global map [7].

Identified Gap: Although previous works demonstrate effective lightweight vision processing, centroid-based tracking, PID control, and sensor-assisted obstacle avoidance, there is a lack of hybrid architectures that seamlessly integrate the spatial geometry of perspective-transformed Dual-ROI vision with the absolute measurement reliability of LiDAR for discrete FSM-based evasion scheduling.

3 Proposed Method

Our system architecture relies on the duality of continuous control loops for lane keeping and discrete state automata for safety transitions.

3.1 Dual-ROI Lane Tracking & Perspective Transform

To reduce color-based vulnerabilities, the input image undergoes grayscale conversion, Gaussian blur, and Canny edge detection [3]. A Perspective Transform warps the image into a "Bird's Eye View" [8]. This aligns the lane markings to be perfectly parallel.

The camera's field of view is split:

- **Proximal ROI (Near):** Computes the target centerline $x_{target} = (x_{left} + x_{right})/2$. A PID controller regulates the cross-track error $e(t) = x_{target} - x_{center}$ for continuous steering adjustments [4].
- **Distal ROI (Far):** Acts as a binary look-ahead trigger during evasion maneuvers.

4 Speed Track Approach 1: LiDAR-Scheduled Orthogonal Evasion

When the vehicle operates in the primary lane tracking state, the front-facing LiDAR continuously monitors the forward sector.

4.1 Evasion Scheduling and Orthogonal Rotation

Upon detecting an obstacle at distance d_{obs} where $d_{obs} \leq D_{threshold}$, the Evasion Scheduler is invoked. The vehicle executes a deterministic orthogonal sequence:

1. **Scheduled Stop:** The vehicle calculates a braking distance to safely halt before the obstacle.
2. **Directional Pivot:** The vehicle executes a strict 90° pivot in place.
3. **Lateral Transit:** The vehicle proceeds in a straight line along this perpendicular heading.

4.2 Far-ROI Triggered Re-alignment

During the lateral transit, the Distal (Far) ROI monitors the upcoming track boundary. When the Distal ROI detects that the track boundary has completely disappeared from its view, the system deduces that the lateral shift is sufficient. This triggers the Re-alignment Phase:

$$\theta_{correction} = -90^\circ \quad (1)$$

The vehicle counter-pivots to neutralize the initial rotation, instantly resetting its heading parallel to the track, and resumes Proximal-ROI lane tracking.

5 Speed Track Approach 2: Trigonometric LiDAR-Based Trajectory Evasion

While Approach 1 relies on strict grid-like movements, Approach 2 proposes a dynamic geometric trajectory. In this state, the vehicle defaults to central lane tracking with a baseline heading of $\theta = 0^\circ$.

5.1 Obstacle Detection and Optimal Evasion Angle

The LiDAR continuously scans the forward 180-degree sector. Upon detecting an anomaly, lane tracking is immediately suspended, and the vehicle is scheduled to come to a complete halt at an adaptive stopping distance from the obstacle. Instead of using a fixed stopping distance, the stopping distance is defined as:

$$d_{stop} = d_{brake} + d_{buffer} + d_{sensor} \quad (2)$$

where d_{brake} , d_{buffer} , and d_{sensor} denote braking distance, minimum safety buffer, and sensor-delay margin, respectively. While stationary, LiDAR estimates the obstacle profile and the camera provides the lane-valid region. A valid sub-goal $P_{sg}(x_{sg}, y_{sg})$ is selected only if it lies in free space, stays inside the black lane, and preserves obstacle and boundary clearance. The optimal non-orthogonal evasion angle α is then computed from the selected sub-goal:

$$\alpha = \tan^{-1} \left(\frac{x_{sg}}{y_{sg}} \right) \quad (3)$$

or equivalently:

$$\alpha = \text{atan2}(x_{sg}, y_{sg}) \quad (4)$$

The computed angle α is accepted only if the predicted vehicle footprint remains within the camera-derived lane-valid region. To ensure that the wheels do not move outside the black lane area, the following lane-boundary constraint must be satisfied:

$$d_{boundary} > \frac{W_{vehicle}}{2} + d_{margin} \quad (5)$$

where $d_{boundary}$ is the distance from the vehicle center to the nearest lane boundary, $W_{vehicle}$ is the vehicle width, and d_{margin} is the required safety margin. After the angle is validated, the vehicle performs an in-place pivot by α using opposite wheel-pair actuation and initiates the evasion transit.

5.2 Clearance Validation and Symmetric Return

The vehicle travels along the vector α . During this transit, the lateral LiDAR rays continuously measure the distance to the obstacle, while the camera continues to monitor the lane boundaries. The obstacle is considered cleared when the side sector remains obstacle-free for multiple consecutive LiDAR frames and the obstacle is no longer detected beside the vehicle. At this checkpoint, the system constructs an isosceles trapezoid geometric model using the initial stopping distance, the evasion angle α , and the lateral clearance distance. Using trigonometric relationships, the system estimates the nominal transit distance L traveled during evasion:

$$L = \sqrt{x_{sg}^2 + y_{sg}^2} \quad (6)$$

This value is treated as a nominal estimate rather than a fixed open-loop distance, since the actual vehicle motion may be affected by wheel slip, sensor delay, and minor heading errors. To return to the original centerline, the system executes a symmetric recovery path:

$$\theta_{return_pivot} = -2\alpha \quad (7)$$

The vehicle pivots by -2α to establish an inward trajectory converging towards the centerline. It then travels an initial recovery distance L_{return} , where:

$$L_{return} \approx L \quad (8)$$

During this recovery motion, the camera continuously measures the centerline error e_{center} . Instead of relying only on the estimated distance L , the vehicle stops the recovery motion when it has sufficiently returned to the centerline:

$$|e_{center}| < \varepsilon \quad (9)$$

where ε is the acceptable centerline error threshold. Upon satisfying this condition, the vehicle executes a final corrective pivot of $+\alpha$ to restore the $\theta = 0^\circ$ heading, and re-engages the lane tracking ROI.

6 Approach 3: Smart City Perception and Event-Driven Intersection Automata

The ‘‘Smart City’’ navigation challenge demands a highly reliable cognitive workflow to handle complex structural road scenes containing traffic signs and signal lights. To preserve computational bandwidth on the edge platform, we formalize this task into a discrete Finite State Machine (FSM) governed by a multi-variable matrix: spatial distance and visual semantic classification.

6.1 State Space Taxonomy and Core Topology

The behavioral control of the vehicle at any intersection is dynamically mapped by monitoring two environmental vectors: Object Distance ($D \in \{\text{Far}, \text{Near}\}$) and Sign Attributes ($S \in \{\text{Position Known}, \text{Type Known}\}$). This combinatoric matrix yields four discrete operational states:

- **State 1 (S_1 : Far + Type Known):** The vehicle is physically distant from the intersection but successfully extracts the classification type or semantic meaning of the signpost.
- **State 2 (S_2 : Near + Position Known):** The vehicle has advanced close to the intersection, but due to severe perspective distortion, glare, or cropping, the camera can only track the physical coordinates/side of the signpost without verifying its category.
- **State 3 (S_3 : Far + Structural Baseline):** The exploratory starting state. The vehicle defaults to continuous PID lane keeping while continuously scanning the look-ahead horizon for upcoming traffic signs or light poles.
- **State 4 (S_4 : Near + Type Known):** The ultimate target state ($\geq 90\%$ spatial probability threshold). The vehicle is positioned precisely at the intersection node boundary, facing the signpost for final validation.

6.2 Automata Transition Mechanics and Memory Registers

The operational workflow is systematically engineered to enforce convergence toward State 4. The structural lifecycle is formalized as follows:

$$S_3 \longrightarrow \{S_1 \text{ or } S_2\} \longrightarrow S_4 \quad (10)$$

As shown in Fig. 1, the vehicle starts from the exploratory baseline state S_3 , then transitions through either far-field semantic recognition (S_1) or near-field positional tracking (S_2), before reaching the final validation state S_4 at the intersection boundary.

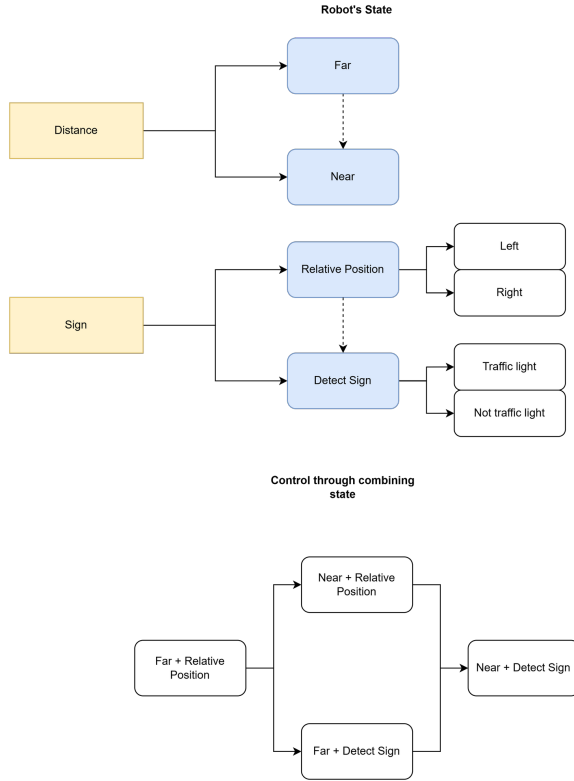


Figure 1: State space transition diagram for Smart City intersection automata.

Each individual state triggers a unique scheduling loop:

In S_3 , the vehicle performs normal lane-tracking operations. Once a peripheral object is identified, the system transitions to either S_1 or S_2 .

When transitioning into S_1 , the system processes the far-field visual data and instantiates a dynamic temporary guidance register, denoted as V_{temp} . The vehicle maintains its lane tracking sequence until the camera and localized topological node-detection algorithms signal arrival at the intersection entry boundary.

If sensory occlusion or illumination noise disrupts classification during the physical approach, the system falls back to S_2 . In this fail-safe phase, the vehicle bypasses real-time classification and forces navigation

commands by extracting the historical direction stored inside V_{temp} .

Conversely, when the system moves from S_1 directly to S_4 , the vehicle executes a pre-programmed spatial realignment, physically pivoting the camera directly toward the bounding coordinates of the signpost. The system executes a near-field classification, cross-referencing the result against V_{temp} . If the outputs match, the vehicle fulfills the instruction. If a structural conflict arises, the real-time classification at S_4 takes supreme priority.

6.3 Perception Multi-Tracking: Traditional Template Matching vs. Quantized YOLOv8-nano

To maximize tactical flexibility, this framework supports two independent perception pathways for sign decoding:

Track A (Traditional Image Arithmetic): Under S_1 , the cropped image patch of the sign is compared against structural template directories pre-saved on the disk (e.g., categorized folders: “No Entry”, “Turn Right”, “Turn Left”). The template yielding the highest normalized cross-correlation score defines the category and populates V_{temp} .

Track B (Modern YOLOv8-nano): Traditional matrix comparison is replaced by a lightweight object detection model. A specialized dataset featuring localized intersection semantics is collected, annotated, and trained. The resulting architecture executes identical state automata transitions under TensorRT quantization.

6.4 Preemptive Traffic Light Handling Hierarchy

Across all operational states, traffic light state registration runs with absolute execution priority. Upon converging at the intersection boundary (S_4), the system enforces a strict hierarchical policy before evaluating any signpost attributes:

- Red Light Assertion:** The FSM instantly overrides steering commands and triggers immediate braking. The vehicle remains locked in a stationary read loop. Sign verification is suppressed until a Green light condition is met.
- Green Light Clearance:** The vehicle instantly transitions to sign-alignment, cross-references the semantic classification with V_{temp} , and executes the directional trajectory.

7 Experimental Plan

To validate the proposed architecture, we establish a rigorous testing framework on the physical Jetson platform.

7.1 Evaluation Metrics & Baseline

- **Frames Per Second (FPS):** Must sustain ≥ 20 FPS to satisfy the scoring requirement, with an internal target of ≥ 25 FPS.
- **Cross-Track Error (CTE):** Measured in pixels, evaluating the stability of the PID controller.
- **Evasion Success Rate (ESR):** Percentage of successful obstacle bypasses without boundary collision.

Baseline Method: A standard pipeline utilizing static OpenCV HSV masking combined with threshold-based LiDAR emergency braking.

7.2 Data Collection & Log Analysis

Telemetry data (Steering angles, PID values, FSM states, and LiDAR point clouds) will be streamed via WebSockets to a local dashboard. Post-run analysis will utilize ROS (Robot Operating System) bag files to graph PID overshoot and tune K_p, K_d parameters offline.

8 Implementation Plan

The project will be executed over a 4-week agile sprint:

- **Week 1:** Camera calibration, Perspective Transform, and PID tuning for Proximal ROI.
- **Week 2:** LiDAR integration and Orthogonal FSM logic (Approach 1).
- **Week 3:** Advanced Trigonometric FSM logic (Approach 2) and Side-LiDAR logic.
- **Week 4:** Full system integration, telemetry dashboard development, and sa-ban stress testing.

Technical Risks & Mitigation: 1) *Sensor Noise:* LiDAR reflection errors. *Mitigation:* Implement a Kalman filter or moving average filter on LiDAR data [9]. 2) *Thermal Throttling:* Overheating CPU/GPU. *Mitigation:* Enforce deterministic CV algorithms to keep utilization below 60%, combined with active cooling.

9 Expected Outcome & Limitation

Expected Outcome: A highly deterministic, low-latency navigation system capable of aggressive lane tracking and mathematically precise obstacle evasion, less dependent on ambient lighting conditions compared with HSV-only pipelines.

Limitations: The method assumes the physical presence of detectable track boundaries (walls or high-contrast lines). If the track lacks distinct visual edges in the Distal ROI, Approach 1's trigger mechanism may fail, necessitating fallback to Approach 2's LiDAR-only measurement.

10 Conclusion

This proposal details a hybrid perception architecture decoupling strategic FSMs from tactical PID control. By mathematically formalizing the evasion trajectories and utilizing Dual-ROI vision, the system guarantees stability and hardware efficiency, providing a robust foundation for the competition.

References

- [1] M. Bojarski et al., "End to end learning for self-driving cars," *arXiv preprint arXiv:1604.07316*, 2016.
- [2] A. Asvadi, A. Garrote, C. Premebida, P. Peixoto, and U. J. Nunes, "Multimodal vehicle detection: fusing 3D-LIDAR and color camera data," *Pattern Recognition Letters*, vol. 115, pp. 20-29, 2018.
- [3] J. Canny, "A computational approach to edge detection," *IEEE Transactions on pattern analysis and machine intelligence*, no. 6, pp. 679-698, 1986.
- [4] K. J. Åström and T. Hägglund, *Advanced PID control*. ISA-The Instrumentation, Systems, and Automation Society, 2006.
- [5] M. Shamoushaki and M. H. Ghasemi, "The Application of Image Moments and Gaussian Blur for Line Detection in Differential-Drive Mobile Robots," *International Journal of Computer Applications*, vol. 186, no. 53, pp. 43-51, Dec. 2024, doi: 10.5120/ijca2024924218.
- [6] R. Farkh and K. Aljaloud, "Vision Navigation Based PID Control for Line Tracking Robot," *Intelligent Automation & Soft Computing*, vol. 35, no. 1, pp. 901-911, 2023, doi: 10.32604/iasc.2023.027614.
- [7] W. Chen, J. Sun, W. Li, and D. Zhao, "A real-time multi-constraints obstacle avoidance method using LiDAR," *arXiv preprint arXiv:2006.00142*, 2020.
- [8] R. Szeliski, *Computer Vision: Algorithms and Applications*, 2nd ed., Springer, 2022.
- [9] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, pp. 35-45, 1960.